

WEEK-4

COURSE : PROBLEM SOLVING AND TESTING USING JAVA

COURSE CODE : 10211CS227

NAME : C. HIMATEJA

VTUNO : 28828

TASKS :

1.DETERMINE IF STRING HALVES ARE ALIKE

2.LAPINDROME

3.COMPARE THE TRIPLETS

4.CONTAIN DUPLICATES

5.TIME CONVERSION

6.MIVE ZEROES

7.DIAGONAL DIFFERENCE

8.TRANSPOSE MATRIX

9.MATRIX BLOCKS SUM

10.MATRIX NOTATION ALGORITHM

1.You are given a string *s* of even length. Split this string into two halves of equal lengths, and let *a* be the first half and *b* be the second half.

Two strings are alike if they have the same number of vowels ('a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'). Notice that *s* contains uppercase and lowercase letters.

Return true if *a* and *b* are alike. Otherwise, return false.

Example 1:

Input: s = "book"

Output: true

Explanation: a = "bo" and b = "ok". a has 1 vowel and b has 1 vowel. Therefore, they are alike.

Example 2:

Input: s = "textbook"

Output: false

Explanation: a = "text" and b = "book". a has 1 vowel whereas b has 2. Therefore, they are not alike.

Notice that the vowel o is counted twice.

Constraints: $2 \leq$

s.length $\leq$ 1000

s.length is even.

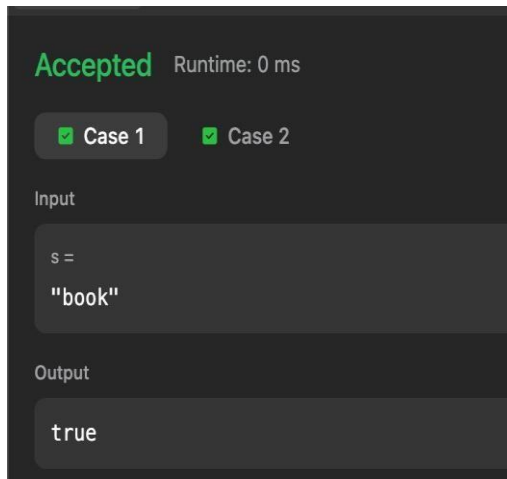
s consists of uppercase and lowercase letters.

Program:

```
class Solution {  
    public boolean halvesAreAlike(String s) {  
        String vowels = "aeiouAEIOU";    int count  
        = 0;    int n = s.length();  
  
        for (int i = 0; i < n / 2; i++) {        if  
(vowels.indexOf(s.charAt(i)) != -1) {  
            count++;  
        }  
  
        if (vowels.indexOf(s.charAt(i + n / 2)) != -1) {  
            count--;  
        }  
    }  
}
```

```
        return count == 0;
    }
}
```

Output:



2. Lapindrome is defined as a string which when split in the middle, gives two halves having the same characters and same frequency of each character. If there are odd number of characters in the string, we ignore the middle character and check for lapindrome. For example *gaga* is a lapindrome, since the two halves *ga* and *ga* have the same characters with same frequency. Also, *abccab*, *rotor* and *xyzxy* are a few examples of lapindromes. Note that *abbaab* is NOT a lapindrome. The two halves contain the same characters but their frequencies do not match.

Your task is simple. Given a string, you need to tell if it is a lapindrome.

Input:

First line of input contains a single integer **T**, the number of test cases.

Each test is a single line containing a string **S** composed of only lowercase English alphabet.

Output:

For each test case, output on a separate line: "YES" if the string is a lapindrome and "NO" if it is not.

Constraints:

- $1 \leq T \leq 100$

- $2 \leq |S| \leq 1000$, where $|S|$ denotes the length of S

Program:

```
import java.util.*;
import java.lang.*;
import java.io.*; class
Codechef {
    public static void main(String[] args) throws java.lang.Exception {
Scanner sc = new Scanner(System.in);    if (!sc.hasNextInt()) return;
int t = sc.nextInt();    while (t-- > 0) {        String s = sc.next();
int len = s.length();
        String left = s.substring(0, len / 2);
String right = s.substring((len + 1)/2;
char[] leftArr = left.toCharArray();        char[]
rightArr = right.toCharArray();
        Arrays.sort(leftArr);
        Arrays.sort(rightArr);
        if (Arrays.equals(leftArr, rightArr)) {
            System.out.println("YES");
        } else {
            System.out.println("NO");
        }
    }
    sc.close();
}
```

Output:

```
6
gaga
abcde
rotor
xyzxy
abbaab
ababc

Your Output
YES
NO
YES
YES
NO
NO
```

3. Alice and Bob each created one problem for HackerRank. A reviewer rates the two challenges, awarding points on a scale from 1 to 100 for three categories: problem clarity, originality, and difficulty.

The rating for Alice's challenge is the triplet $a = (a[0], a[1], a[2])$, and the rating for Bob's challenge is the triplet $b = (b[0], b[1], b[2])$.

The task is to calculate their comparison points by comparing each category:

If $a[i] > b[i]$, then Alice is awarded 1 point.

If $a[i] < b[i]$, then Bob is awarded 1 point.

If $a[i] = b[i]$, then neither person receives a point. Example

$a = [1, 2, 3]$ b

$= [3, 2, 1]$

For elements $*0*$, Bob is awarded a point because $a[0] < b[0]$.

For the equal elements $a[1]$ and $b[1]$, no points are earned.

Finally, for elements 2, $a[2] > b[2]$ so Alice receives a point.

The return array is $[1, 1]$ with Alice's score first and Bob's second.

Function Description

Complete the function `compareTriplets` with the following parameter(s):

`int a[3]`: Alice's challenge rating `int`

`b[3]`: Bob's challenge rating Returns

`int[2]`: the first element is Alice's score and the second is Bob's score

Input Format

The first line contains 3 space-separated integers, $a[0]$, $a[1]$, and $a[2]$, the respective values in triplet a .

The second line contains 3 space-separated integers, $b[0]$, $b[1]$, and $b[2]$, the respective values in triplet b .

Constraints

$$1 \leq a[i] \leq 100$$

$$1 \leq b[i] \leq 100$$

Sample Input 0:

5 6 7

3 6 10

Sample Output 0:

1 1

Program:

```
import java.io.*; import java.math.*; import java.security.*; import java.text.*;
import java.util.*; import java.util.concurrent.*; import java.util.function.*;
import java.util.regex.*; import java.util.stream.*; import static
java.util.stream.Collectors.joining; import static java.util.stream.Collectors.toList;
class Result {    public static List<Integer> compareTriplets(List<Integer> a,
List<Integer> b) { int alice = 0;    int bob = 0;    for (int i = 0; i < 3; i++) {        if
(a.get(i) > b.get(i)) {            alice++;
        } else if (a.get(i) < b.get(i)) {
bob++;
        }
    }

    List<Integer> result = new ArrayList<>();
result.add(alice);    result.add(bob);

return result;
```

```

    }
}

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        List<Integer> a = Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split("
"))

            .map(Integer::parseInt)

            .collect(toList());

        List<Integer> b = Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split("
"))

            .map(Integer::parseInt)

            .collect(toList());

        List<Integer> result = Result.compareTriplets(a, b);

bufferedWriter.write(        result.stream()

            .map(Object::toString)

            .collect(joining(" "))

            + "\n"

        );

        bufferedReader.close();

bufferedWriter.close();

    }

}

```

Output:

Sample Test case 0

Sample Test case 1

Input (stdin)

```
1 5 6 7
2 3 6 10
```

Your Output (stdout)

```
1 1 1
```

Expected Output

```
1 1 1
```

4. Given an integer array `nums`, return `true` if any value appears at least twice in the array, and return `false` if every element is distinct.

Example 1:

Input: `nums = [1,2,3,1]`

Output: `true`

Explanation:

The element 1 occurs at the indices 0 and 3.

Example 2:

Input: `nums = [1,2,3,4]`

Output: `false` Explanation:

All elements are distinct.

Example 3:

Input: `nums = [1,1,1,3,3,4,3,2,4,2]`

Output: `true`

Constraints:

$1 \leq \text{nums.length} \leq 105$

$-109 \leq \text{nums}[i] \leq 109$

Program: import

java.util.HashSet; class

Solution {

```
    public boolean containsDuplicate(int[] nums) {
```

```
        HashSet<Integer> set = new HashSet<>();    for
```

```
        (int num : nums) {        if (set.contains(num)) {
```

```
            return true;
```

```
        }
```

```
        set.add(num);
```

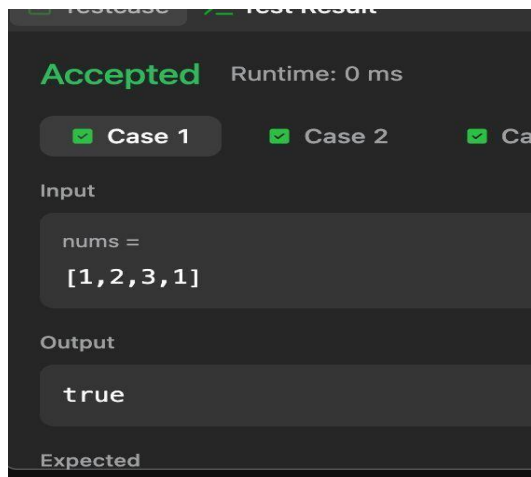
```
    }
```

```
    return false;
```

```
}
```

```
}
```

Output:



5. Given a time in -hour AM/PM format, convert it to military (24-hour) time.

Note: - 12:00:00AM on a 12-hour clock is 00:00:00 on a 24-hour clock.

- 12:00:00PM on a 12-hour clock is 12:00:00 on a 24-hour clock.

Example

Return '12:01:00'.

Return '00:01:00'.

Function Description

Complete the function with the following parameter(s):

: a time in hour format

Returns

: the time in hour format

Input Format

A single string that represents a time in -hour clock format (i.e.: or).

Constraints

All input times are valid

Sample Input 0

07:05:45PM

Sample Output 0

19:05:45

Program:

```
import java.io.*; import
java.math.*; import
java.security.*; import
java.text.*; import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static
```

```

java.util.stream.Collectors.join
ing; import static
java.util.stream.Collectors.toLi
st;

class Result {    public static String
timeConversion(String s) {

    // Extract the period (AM or PM) from the last two characters
    String period = s.substring(s.length() - 2);

    // Extract the numerical time portion (hh:mm:ss)
    String time = s.substring(0, s.length() - 2);

    // Split into components: [hours, minutes, seconds]
    String[] parts = time.split(":");    int hours =
    Integer.parseInt(parts[0]);    if
    (period.equalsIgnoreCase("AM")) {

        // Special case: 12 AM becomes 00 in 24-hour time
        if (hours == 12) {
hours = 0;

        }

    } else {

        // Special case: 12 PM stays 12; all other PM hours add 12
if (hours != 12) {        hours += 12;

        }

    }

    // Format hours back to 2 digits with leading zero if necessary    return
    String.format("%02d:%s:%s", hours, parts[1], parts[2]); }

}

```

```

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH")));

        String s = bufferedReader.readLine();

        String result = Result.timeConversion(s);

        bufferedWriter.write(result);

        bufferedWriter.newLine();

        bufferedReader.close();

        bufferedWriter.close();

    }

}

```

Output:

Test case 0	
Input (stdin)	
1	07:05:45PM
Your Output (stdout)	
1	19:05:45
Expected Output	
1	19:05:45

6. Given an integer array `nums`, move all 0's to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

Input: nums = [0,1,0,3,12]

Output: [1,3,12,0,0] Example

2:

Input: nums = [0]

Output: [0]

Constraints:

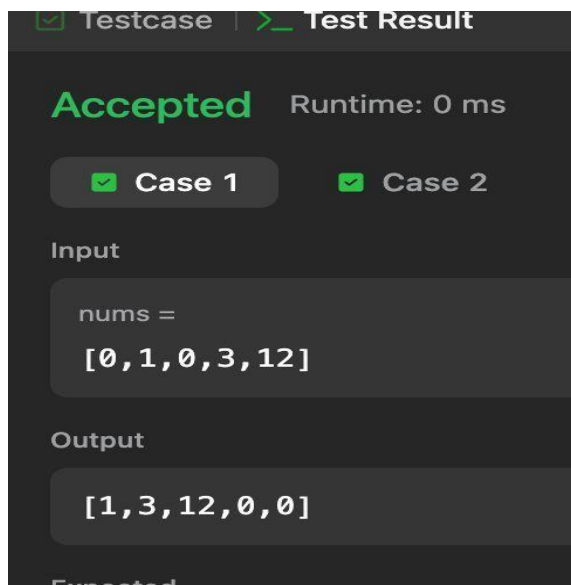
$1 \leq \text{nums.length} \leq 104$

$-231 \leq \text{nums}[i] \leq 231 - 1$

Program:

```
class Solution {    public void
moveZeroes(int[] nums) {
    int index = 0;
    // Move all non-zero elements to the front
    for (int i = 0; i < nums.length; i++) {        if
(nums[i] != 0) {            nums[index] = nums[i];
index++;
        }
    }
    // Fill the remaining positions with 0
    while (index < nums.length) {
nums[index] = 0;        index++;
    }
}
}
```

Output:



7. Given a square matrix, calculate the absolute difference between the sums of its diagonals.

For example, the square matrix is shown below:

1 2 3

4 5 6

9 8 9

The left-to-right diagonal = .

The right-to-left diagonal = .

Their absolute difference is .

Function description

Complete the function with the following parameter:

: a 2-D array of integers

Return

: the absolute difference in sums along the diagonals

Input Format

The first line contains a single integer, , the number of rows and columns in the square matrix .

Each of the next lines describes a row, , and consists of space-separated integers . STDIN

Function

3 arr[][] sizes n = 3, m = 3

11 2 4 arr = [[11, 2, 4], [4, 5, 6], [10, 8, -12]]

4 5 6

10 8 -12

Sample Output

15

Explanation

The primary diagonal is:

11

5

-12

Sum across the primary diagonal: .

The secondary diagonal is:

4

5 10 Sum across the secondary diagonal:

Difference:

Note: $|x|$ is the absolute value of x.

Program:

```
import java.io.*; import java.math.*; import
java.security.*; import java.text.*; import
java.util.*; import java.util.concurrent.*; import
java.util.function.*; import java.util.regex.*;
import java.util.stream.*; import static
java.util.stream.Collectors.joining; import static
java.util.stream.Collectors.toList;
```

```

class Result {
    public static int
    diagonalDifference(List<List<Integer>> arr) {
        int primary = 0;
        int secondary = 0;
        int n = arr.size();
        for (int i = 0; i < n; i++) {
            primary += arr.get(i).get(i);
            secondary += arr.get(i).get(n - 1 - i);
        }
        return Math.abs(primary - secondary);
    }
}

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new
        InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        int n = Integer.parseInt(bufferedReader.readLine().trim());

        List<List<Integer>> arr = new ArrayList<>();

        IntStream.range(0, n).forEach(i -> {
            try {
                arr.add(
                    Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))
                        .map(Integer::parseInt)
                        .collect(toList())
                );
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });
    }
}

```

```

        int result = Result.diagonalDifference(arr);
        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();
        bufferedReader.close();    bufferedWriter.close();
    }
}

```

Output:

Test case 0	
Input (stdin)	
1	3
2	11 2 4
3	4 5 6
4	10 8 -12
Your Output (stdout)	
1	15
Expected Output	
1	15

8. Given a 2D integer array matrix, return the transpose of matrix.

The transpose of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.

Example 1:

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]

Output: [[1,4,7],[2,5,8],[3,6,9]] Example

2:

Input: matrix = [[1,2,3],[4,5,6]]

Output: [[1,4],[2,5],[3,6]] Constraints:

m == matrix.length n

== matrix[i].length 1

<= m, n <= 1000

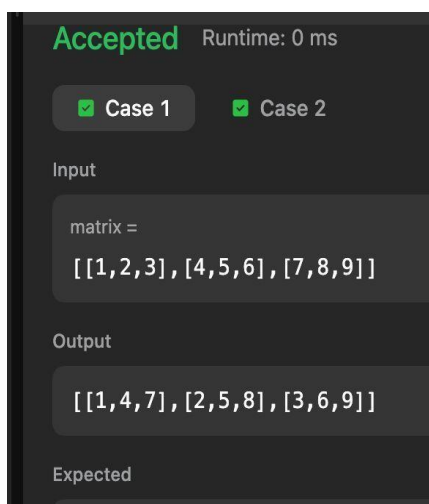
$1 \leq m * n \leq 105$

$-109 \leq \text{matrix}[i][j] \leq 109$

Program:

```
class Solution {    public int[][]  
    transpose(int[][] matrix) { int m =  
    matrix.length;  
  
        int n = matrix[0].length;  
    int[][] result = new int[n][m];  
    for (int i = 0; i < m; i++) {  
    for (int j = 0; j < n; j++) {  
    result[j][i] = matrix[i][j];  
        }  
    }  
    return result;  
    }  
}
```

Output:



9. Given a $m \times n$ matrix `mat` and an integer `k`, return a matrix `answer` where each `answer[i][j]` is the sum of all elements `mat[r][c]` for:

$i - k \leq r \leq i + k, j - k$

$\leq c \leq j + k$, and

(r, c) is a valid position in the matrix.

Example 1:

Input: `mat = [[1,2,3],[4,5,6],[7,8,9]]`, `k = 1` Output:

`[[12,21,16],[27,45,33],[24,39,28]]` Example 2:

Input: `mat = [[1,2,3],[4,5,6],[7,8,9]]`, `k = 2`

Output: `[[45,45,45],[45,45,45],[45,45,45]]`

Constraints: `m == mat.length` `n ==`

`mat[i].length`

`1 <= m, n, k <= 100`

`1 <= mat[i][j] <= 100`

```
Program: class Solution {    public int[][]
matrixBlockSum(int[][] mat, int k) {        int m =
mat.length;        int n = mat[0].length;        int[][]
prefix = new int[m + 1][n + 1];        // Build prefix
sum matrix        for (int i = 1; i <= m; i++) {
for (int j = 1; j <= n; j++) {            prefix[i][j] =
mat[i - 1][j - 1]
+ prefix[i - 1][j]
+ prefix[i][j - 1]
- prefix[i - 1][j - 1];
        }
    }
```

```

int[][] answer = new int[m][n];

// Calculate block sums
for (int i = 0; i < m; i++) {
    for (int
j = 0; j < n; j++) {
        int r1 =
Math.max(0, i - k);
        int c1 =
Math.max(0, j - k);
        int r2 =
Math.min(m - 1, i + k);
        int c2 =
Math.min(n - 1, j + k);
        answer[i][j]
= prefix[r2 + 1][c2 + 1]
- prefix[r1][c2 + 1]
- prefix[r2 + 1][c1]
+ prefix[r1][c1];
    }
}

return answer;
}
}

```

Output:

```

Input
mat =
[[1,2,3],[4,5,6],[7,8,9]]

k =
1

Output
[[12,21,16],[27,45,33],[24,39,28]]

Expected

```

10. You are given a 2D matrix of dimension $m \times n$ and a positive integer r . You have to rotate the matrix r times and print the resultant matrix. Rotation should be in anti-clockwise direction.

Rotation of a matrix is represented by the following figure. Note that in one rotation, you have to shift elements by one step only.

matrix-rotation

It is guaranteed that the minimum of m and n will be even.

As an example rotate the Start matrix by 2:

Start	First	Second
1 2 3 4	2 3 4 5	3 4 5 6
12 1 2 5	-> 1 2 3 6	-> 2 3 4 7
11 4 3 6	12 1 4 7	1 2 1 8
10 9 8 7	11 10 9 8	12 11 10 9

Function Description

Complete the `matrixRotation` function in the editor below.

`matrixRotation` has the following parameter(s):

`int matrix[m][n]`: a 2D array of integers

`r`: the rotation factor

Prints

It should print the resultant 2D integer array and return nothing. Print each row on a separate line as space-separated integers.

Input Format

The first line contains three space separated integers, m , n , and r , the number of rows and columns in $matrix$, and the required rotation.

The next m lines contain n space-separated integers representing the elements of a row of $matrix$.

Sample Input #01

STDIN Function

4 4 2 rows m = 4, columns n = 4, rotation factor r = 2

1 2 3 4 matrix = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]]

5 6 7 8

9 10 11 12

13 14 15 16

Sample Output #01

3 4 8 12

2 11 10 16

1 7 6 15

5 9 13 14

Explanation #01

The matrix is rotated through two rotations.

1 2 3 4 2 3 4 8 3 4 8 12

5 6 7 8 1 7 11 12 2 11 10 16

9 10 11 12 -> 5 6 10 16 -> 1 7 6 15

13 14 15 16 9 13 14 15 5 9 13 14

Sample Input #0

5 4 7

1 2 3 4

7 8 9 10

13 14 15 16

19 20 21 22

25 26 27 28

Sample Output #02

28 27 26 25

22 9 15 19

16 8 21 13

10 14 20 7

4 3 2 1

Program:

```
import java.io.*; import java.math.*; import java.security.*; import
java.text.*; import java.util.*; import java.util.concurrent.*; import
java.util.function.*; import java.util.regex.*; import java.util.stream.*;
import static java.util.stream.Collectors.joining; import static
java.util.stream.Collectors.toList; class Result {    public static void
matrixRotation(List<List<Integer>> matrix, int r) {    int m =
matrix.size();    int n = matrix.get(0).size();    int[][] ans = new
int[m][n];    int layers = Math.min(m, n) / 2;    for (int layer = 0;
layer < layers; layer++) {        List<Integer> list = new ArrayList<>();

        // Top        for (int j = layer; j < n
- layer; j++)
list.add(matrix.get(layer).get(j));

        // Right        for (int i = layer + 1; i < m -
layer - 1; i++)
list.add(matrix.get(i).get(n - layer - 1));

        // Bottom        for (int j = n - layer - 1; j
>= layer; j--)        list.add(matrix.get(m -
layer - 1).get(j));
```

```

        // Left        for (int i = m - layer - 2; i >
layer; i--)
list.add(matrix.get(i).get(layer));        int size
= list.size();        int rot = r % size;        int
idx = rot;        // Top        for (int j = layer;
j < n - layer; j++)        ans[layer][j] =
list.get((idx++) % size);

        // Right        for (int i = layer + 1; i < m -
layer - 1; i++)        ans[i][n - layer - 1] =
list.get((idx++) % size);

        // Bottom        for (int j = n - layer - 1; j >=
layer; j--)        ans[m - layer - 1][j] =
list.get((idx++) % size);

        // Left        for (int i = m - layer - 2; i >
layer; i--)        ans[i][layer] =
list.get((idx++) % size);    }

    for (int i = 0; i < m; i++) {
for (int j = 0; j < n; j++) {
System.out.print(ans[i][j]);

    if (j != n - 1)

        System.out.print(" ");

    }

    System.out.println();

}

}

}

```

```

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));

        String[] firstMultipleInput = bufferedReader.readLine().replaceAll("\\s+$", "").split("
");

        int m = Integer.parseInt(firstMultipleInput[0]);
int n = Integer.parseInt(firstMultipleInput[1]);    int
r = Integer.parseInt(firstMultipleInput[2]);

        List<List<Integer>> matrix = new ArrayList<>();
IntStream.range(0, m).forEach(i -> {        try {

            matrix.add(

                Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))

                    .map(Integer::parseInt)

                    .collect(toList())

            );

        } catch (IOException ex) {
throw new RuntimeException(ex);

        }

    });

        Result.matrixRotation(matrix, r);

        bufferedReader.close();

    }

}

```

Output:

le Test case 0

le Test case 1

le Test case 2

le Test case 3

Input (stdin)

```
1 4 4 1
2 1 2 3 4
3 5 6 7 8
4 9 10 11 12
5 13 14 15 16
```

Your Output (stdout)

```
1 2 3 4 8
2 1 7 11 12
3 5 6 10 16
4 9 13 14 15
```